What's Next?

You now know how to build dynamic, server-powered bridge components for Android. Adding a new native button is now as easy as adding one or two `data-*` attributes to your HTML. And because you made the title and image dynamic, your options for customization are endless.

You can follow the technique you learned in this chapter to build your own components, taking advantage of any native API. Swap out the code in `onClick` to execute whatever Kotlin you'd like, such as accessing the Health Platform API,[51] interacting with the device's sensors,[52] and more.

If you're looking for more examples or inspiration, check out my open source bridge component library.[53] It includes a bunch of generalized, production-ready bridge components extracted from real-world client projects. Once configured, each component can be added to any page of your app and customized with a bit of HTML.

With bridge components done, your app is almost feature-complete! All that's left is adding push notifications in Chapter 10, *Send Push Notifications with APNs and FCM*. But to do that, we first need to register our apps with Apple and Google to use their push notification services.

In the next chapter, you'll learn how to get your app off the emulator and onto a real device. We'll set up the app on App Store Connect and the Google Play Console, package a build in Xcode and Android Studio, and then deploy it with TestFlight and Google Play Testing. And the best part? You can invite your friends and colleagues to download the app and show them what you've been working on.

Footnotes

[48] https://kotlinlang.org/docs/serialization.html